מקרא:
טקסט המסומן בצבע ירוק – הרחבות למי שמעוניין בכך.
החומר המתואר בחלקים אלו אינו מהווה חלק מהחומר המחייב לבחינה.

File I/O מעשי: קריאה, כתיבה

תוכן עניינים:

- **I/O בסיסי:** `read()`, `write()`, *Partial I/O*.

- **היררכיית ה-I/O ו-Buffering**

- **ניהול הקובץ:** `lseek()`, `pread()`, `pwrite()`, `stat()`, `truncate()`, כולל דוגמאות שימוש.

I/O בסיסי: קריאה וכתיבה באמצעות קריאות מערכת

סעיף זה מתמקד באופן שבו תהליך מקיים אינטראקציה ישירה עם ליבת מערכת ההפעלה (Kernel) לצורך העברת נתונים. היינו, לצורך קריאת מידע מקובץ ולצורך כתיבת מידע לקובץ.

קריאות המערכת **`read()`** ו- **`write()`** הן אבני היסוד של פעולות I/O.

קריאות מערכת אלו 'עושות שימוש' (מקבלות כארגומנט) ב- **File Descriptor (FD)**. ה- FDs הם מטיפוס int.
הם לא בטיפוס הנתונים FILE*, בו אתם 'רגילים' לעשות שימוש בהקשר של הפונקציות הסטנדרטיות לעבודה עם קבצים מתוך תכניות C.

קריאת נתונים באמצעות `read()`

תחביר: `ssize_t read(int fd, void *buf, size_t count) ;`

© יצחק נודלר

ניתוח ערכי החזרה וטיפול ב-Partial I/O (קריאה חלקית)

ערך ההחזרה של `read()` הוא קריטי ומחייב טיפול מדויק:

1. **ערך חיובי (`>0`):** מספר הבתים שנקראו בפועל. מצב זה יכול להיות **Partial I/O** (פחות מהבקשה המקורית).
2. **אפס (`0`):** הגיע לסוף הקובץ (**End-of-File - EOF**).
3. **שלילי (`-1`):** אירעה שגיאה. יש לבדוק את משתנה `errno`.

טיפול ב-Partial I/O: יש צורך לכתוב **לולאת עטיפה (Wrapper)** המבצעת קריאות חוזרות ונשנות (Retry) עד שכל מספר הבתים הנדרש נקרא.

טיפול בשגיאות Syscall נפוצות

1. **`EINTR` (Interrupted Call - שיחה שהופרעה):**
   - **מהות:** שגיאה זו מתרחשת כאשר ה-Kernel מקבל **Signal אסינכרוני** (כגון SIGINT או SIGUSR1) בזמן שהתהליך **חסום (Blocked)** בתוך קריאת המערכת (`read()`) וממתין להשלמת ה- I/O. ה-Kernel מפסיק את קריאת המערכת ומחזיר `-1` עם `errno` השווה ל-`EINTR`.
   - **הטיפול הנכון:** עצירת הקריאה אינה מעידה על כשל קבוע. הטיפול הנכון מבחינת המתכנת הוא **לבצע Retry (לולאה)** על הקריאה, תוך התעלמות מ-`EINTR`, כדי שה-I/O יושלם כמתוכנן.

2. **`EWOULDBLOCK` / `EAGAIN` (Non-blocking I/O):**
   - **מהות:** שגיאות אלו מתרחשות **רק** כאשר ה- FD נפתח במצב **Non-blocking** (כלומר, עם הדגל `O_NONBLOCK`). משמעותן היא שאין נתונים זמינים מיידית לקריאה (או שה- Buffer מלא לכתיבה). ה-Kernel **מסרב לחסום** את התהליך ומחזיר שגיאה.
   - **הצורך:** התהליך מקבל חיווי מיידי על מצב ה-I/O ויכול להמשיך לעבודה אחרת, במקום לחכות. שגיאות אלו דורשות טיפול מתקדם (לרוב באמצעות `select()` או `poll()`).

כתיבת נתונים באמצעות write()

תחביר: ssize_t write(int fd, const void *buf, size_t count);

טיפול ב- Partial Writes (כתיבה חלקית)

בדומה ל-read(), גם הקריאה ל-write() יכולה לכתוב פחות בתים מהמבוקש. יש צורך לטפל בהן באמצעות לולאה (Retry) כדי להבטיח **שלמות הנתונים**.

© יצחק גודלר

היררכיית ה-I/O ו-Buffering

הרציונל: עלות קריאת מערכת וביצועים

ההפרדה בין פונקציות ה-Kernel (`read`, `write`) לבין פונקציות ספריית C (`fread`, `fwrite`) היא מימוש של ארכיטקטורה ממוטבת [= optimized] המונעת על ידי שיקולי **Latency** **ועלות**:

1. **עלות Syscall (קריאת מערכת):**
   - **Mode Switch:** קריאה ל-Syscall דורשת **Mode Switch** (מעבר רמת הרשאה) מ- User Mode ל- Kernel Mode.
   - **Context Switch:** אם ה-Kernel קובע שהתהליך צריך **לחכות** ל-I/O, הוא מבצע **Context Switch**. פעולה זו יקרה משום שהיא כרוכה ב**שמירה (Saving) ושחזור (Restoring)** של ה- Context המלא של התהליך בזיכרון (Registers, Stack Pointers).
   - **השפעה על ביצועים:** ה-Context Switch הוא ה-**Overhead (תקורה)** המשמעותי ביותר. ביצוע קריאות `read` רבות מוביל לביצועים גרועים, שכן זמן ה-I/O נשלט על ידי ה- Latency של המעברים היקרים, ולא על ידי מהירות ה- NVM.
2. **Buffer Cache (מטמון הליבה):**
   - **מהות:** ה- **Kernel Page Cache** ב- RAM קיים כדי להימנע מגישה איטית ל- **NVM** (הדיסק).
   - **פעולה:** קריאה ל- `write()` רק מעתיקה את הנתונים ל- Page Cache (ומסמנת אותם כ-"Dirty"). הליבה חוזרת מיידית ל-User Mode, <u>והכתיבה הפיזית לדיסק (Writeback) מתבצעת מאוחר יותר באופן אסינכרוני</u>.

פונקציות הספרייה (libc): Buffering ב- User Space

ספריית C (`libc`) מיישמת שכבת מטמון נוספת כדי להתמודד עם ה-Overhead של ה- Syscalls:

1. **קשר בין `FILE*` ל- FD:**
   - **`fopen()`:** קוראת פנימית ל- `open()` (= מבצעת Syscall) ומקבלת **FD**. מצביע ה- **`FILE*`** הוא מבנה מנוהל על ידי `libc` והוא **מכיל את ה- FD הפנימי** שמשמש לגישה ל-Kernel.
   - **Buffer פנימי:** מבנה ה- `FILE*` מקצה גם **User Buffer** (זיכרון RAM ב- User Space) וכן מנהל את **המיקום הנוכחי** בתוך Buffer זה.

© יצחק נודלר

2. **`fread()` ו- `fwrite()` (I/O לוגי מול פיזי):**
   - **I/O לוגי:** פונקציות אלו קוראות וכותבות ל- **User Buffer באופן לוגי**. כלומר, פעולה מול ה- Buffer ב- User Space, ללא גישה ישירה ל-Kernel. אם המידע נמצא ב-Buffer, אין צורך ב- Syscall.
   - **הפעלת Syscall:** קריאת המערכת `read()` או `write()` מופעלת **רק** כאשר ה- User Buffer **מתרוקן** (ב- `fread`) או **מתמלא** (ב- `fwrite`). זהו הפתרון לבעיית העלות: במקום 100 קריאות Syscall קטנות, מתבצעת קריאה Syscall **אחת גדולה ויעילה**.

3. **ניקוז מטמון (`fflush()`):**
   - **משמעות:** קריאה זו מכריחה כתיבה מיידית (**Buffer Flush**) של כל הנתונים הנמצאים כרגע ב- **User Buffer** אל ה- **Kernel Page Cache** (באמצעות קריאה ל- `write()`).
   - **חשיבות:** מבטיח שנתונים שנכתבו לוגית יגיעו לפחות ל-Kernel לפני כשל אפשרי של התוכנית.

פונקציות עיבוד פורמט (Formatting)

פונקציות אלו מטפלות בהמרה בין **פורמט טקסטואלי** (ASCII) לבין **פורמט בינארי** (של משתני C).

- **הקשר להיררכיה:**
  - **`fprintf()` ל- `fwrite()` ל- `write()` (היינו: Syscall)**
  - **`fscanf()` ל- `fread()` ל- read() (היינו: Syscall)**

רמה	פונקציית קריאה	פונקציית כתיבה	שירות עיקרי
Kernel (L0)	`read()`	`write()`	העברת בתים; עדכון File Offset.
Libc (L1)	`fread()`	`fwrite()`	**Buffering** (מטמון I/O); מזעור מס' ה- Syscalls.
Libc (L2)	`fscanf()`	`fprintf()`	**Formatting** ועיבוד פורמט.

© יצחק נודלר

ניהול הקובץ: מיקום, גודל ומטא-דאטה

שינוי מיקום באמצעות lseek()

lseek() מאפשר לשנות את שדה ה- **File Offset** (המיקום הנוכחי לקריאה/כתיבה) במבנה ה- File Structure.

- **חשיבות:** הופך את הגישה לקובץ מ**טורית** ל**אקראית** (Random Access), החיוני למימוש **בסיסי נתונים** או קובצי אינדקס.
- **יצירת Hole:** העברת ה- Offset מעבר לגודל הקובץ הנוכחי גורמת ליצירת "**חור**" **(Hole)** בקובץ, שאינו תופס מקום על הדיסק בפועל.

קריאה וכתיבה עמדתית [= positional] : pread() ו- pwrite()

פונקציות אלו מאפשרות לבצע קריאה או כתיבה ממיקום נתון **מבלי לשנות** את ה-File Offset הקבוע של הקובץ.

- **אטומיות (Atomic Update) של Metadata:**
  - **מהות:** pread() ו- pwrite() משלבות את פעולת ה-lseek() וה- I/O לתוך קריאת מערכת **אחת**. ה- Kernel מבטיח שכל הפעולה תתבצע כיחידה **אטומית** שאינה יכולה להיות מופרעת על ידי תהליכים אחרים המשתפים את אותו קובץ.
  - **שימוש מעשי:** חיוני **לעדכון שדות מטא-דאטה** (כגון מונה) בראש קובץ אינדקס. כאשר תהליך משתמש ב-pwrite(), הוא מבטיח שאף תהליך אחר לא יזיז את ה-Offset במקביל.

גישה למטא-דאטה באמצעות stat() ו-fstat()

פונקציות אלו מחזירות מבנה המכיל את כל ה-Metadata של הקובץ (גודל, זמני שינוי, הרשאות, UID/GID, Inode Number).

- **שימוש משמעותי:**
  - **תכנית ה- ls:** משתמשת ב-stat() כדי לקבוע את **סוג הקובץ**, גודלו, וזמני הגישה והשינוי שלו.
  - **כלי גיבוי (Backup):** מערכות גיבוי משתמשות בשדה **mtime** (זמן השינוי האחרון של התוכן) כדי לקבוע האם הקובץ השתנה מאז הגיבוי האחרון, ובכך לייעל גיבויים מצטברים/דיפרנציאליים.

© יצחק נודלר

קיטוע קבצים (truncate() ו- ftruncate())

פונקציות אלו מאפשרות לשנות את גודלו של קובץ פעיל.

- **חשיבות: מערכות Log File (יומן)** או **קובץ Swap** משתמשות ב- truncate() כדי **לאפס** את הקובץ לגודל אפס או לגודל חדש, מבלי לבצע את הפעולה האיטית של מחיקה וייצור מחדש של הקובץ.

© יצחק גודלר